МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»
Центр образовательных программ топ-уровня в сфере
информационных технологий Образовательного центра Института №8

Лабораторная работа №3 по курсу

«Операционные системы и архитектура компьютера»

«Управление потоками в ОС. Обеспечение синхронизации между потоками»

Студент: Цыганок Юлия Дмитриевна
Группа: М8О–205БВ–25
Преподаватель: Филимонов Н. С.
Оценка: _____
Дата: _____
Подпись: _____

Москва, 2026

1. ВВЕДЕНИЕ

1.1 Цель работы

Целью является приобретение практических навыков в управлении потоками в операционной системе, обеспечении синхронизации между потоками и исследовании зависимости ускорения и эффективности алгоритма от входных данных и количества потоков.

1.2 Задание

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы. Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска программы.

Вариант 20: Дан массив координат x, y, z. Необходимо найти три точки, которые образуют треугольник максимальной площади.

2. ХОД РАБОТЫ

2.1 Описание варианта и логики работы

Требуется найти треугольник максимальной площади в трехмерном пространстве. Площадь треугольника по трем точкам вычисляется через векторное произведение векторов. Так как количество возможных троек точек равно n3, алгоритм имеет кубическую сложность. Для распараллеливания используется разбиение внешнего цикла перебора первой точки треугольника. Каждый поток обрабатывает свой набор индексов с шагом, равным количеству потоков. Каждый поток находит локальный максимум площади, после чего главный поток сравнивает локальные максимумы и находит глобальный.

2.2 Используемые системные вызовы

Для реализации использовалась библиотека pthread в среде Linux:

- pthread_create() – создание нового потока.
- pthread_join() – ожидание завершения потока главным потоком.
- clock_gettime() – замер времени выполнения.

2.3 Методика выполнения работы

Для выполнения работы использовалась операционная система Linux подсистема WSL 2 в Windows 11, дистрибутив Ubuntu. Компиляция выполнялась компилятором GCC с ключами -pthread и -lm.

Была написана программа triangle.c. Программа принимает два аргумента командной строки:

1. Количество потоков.
2. Количество генерируемых точек.

2.4 Описание работы программы

1. Программа считывает аргументы командной строки (число потоков и точек). Выделяет память и генерирует массив случайных точек с координатами от 0 до 100.
2. Создается массив потоков. Каждому потоку передается структура с указателем на массив точек, общим количеством точек, начальным индексом и шагом.
3. Потоки параллельно перебирают тройки точек и ищут максимальную площадь.
4. Главный поток с помощью pthread_join дожидается завершения всех дочерних потоков.
5. Главный поток сравнивает локальные максимумы и выводит глобальный максимум, координаты искомых точек и затраченное время

3. ЭКСПЕРИМЕНТАЛЬНАЯ ЧАСТЬ

3.1 Доказательство использования потоков

Для доказательства того, что программа действительно использует несколько потоков, был использован системный монитор ps с флагом -L. Пример использования приведен на рис.1.

```
student@DESKTOP-23PR51U:~$ ps -eLf | grep triangle
student      587     360     587  0    9 15:34 pts/0    00:00:00 ./triangle -t 8 -n 15000
student      587     360     588 99    9 15:34 pts/0    00:00:51 ./triangle -t 8 -n 15000
student      587     360     589 99    9 15:34 pts/0    00:00:51 ./triangle -t 8 -n 15000
student      587     360     590 99    9 15:34 pts/0    00:00:51 ./triangle -t 8 -n 15000
student      587     360     591 99    9 15:34 pts/0    00:00:51 ./triangle -t 8 -n 15000
student      587     360     592 99    9 15:34 pts/0    00:00:51 ./triangle -t 8 -n 15000
student      587     360     593 99    9 15:34 pts/0    00:00:51 ./triangle -t 8 -n 15000
student      587     360     594 99    9 15:34 pts/0    00:00:51 ./triangle -t 8 -n 15000
student      587     360     595 99    9 15:34 pts/0    00:00:51 ./triangle -t 8 -n 15000
student      729     601     729  0    1 15:35 pts/2    00:00:00 grep --color=auto triangle
student@DESKTOP-23PR51U:~$
```

Рис. 1 – Использование многопоточности

3.2 Проверка корректности

При запуске с одинаковым количеством точек на 1 и на 4 потоках, программа выдала одинаковую максимальную площадь и одинаковые координаты точек. Это подтверждает правильность распараллеливания.

3.3 Исследование зависимости времени от количества потоков

Как видно из графика, при увеличении числа потоков время выполнения падает, однако ускорение не является линейным. На 2 потоках ускорение составило 2.07.На 4 потоках ускорение составило 3.06, а на 8 потоках — 3.66. Снижение эффективности связано с наличием последовательной части программы и накладными расходами на переключение контекста, когда количество потоков превышает количество физических ядер процессора. Оптимальным для данной задачи и данной машины является использование 4 потоков. Результат показан на рис. 2.

```
student@DESKTOP-23PR51U:/mnt/c/Users/Admin/Desktop/MAI_OS/lab3$ ./triangle -t 1 -n 2000
Thr: 1, N: 2000
Max area: 7449.632038
Points: 103 388 1520
Time: 2.246511 sec
student@DESKTOP-23PR51U:/mnt/c/Users/Admin/Desktop/MAI_OS/lab3$ ./triangle -t 2 -n 2000
Thr: 2, N: 2000
Max area: 7449.632038
Points: 103 388 1520
Time: 1.084550 sec
student@DESKTOP-23PR51U:/mnt/c/Users/Admin/Desktop/MAI_OS/lab3$ ./triangle -t 4 -n 2000
Thr: 4, N: 2000
Max area: 7449.632038
Points: 103 388 1520
Time: 0.734376 sec
student@DESKTOP-23PR51U:/mnt/c/Users/Admin/Desktop/MAI_OS/lab3$ ./triangle -t 8 -n 2000
Thr: 8, N: 2000
Max area: 7449.632038
Points: 103 388 1520
Time: 0.614115 sec
student@DESKTOP-23PR51U:/mnt/c/Users/Admin/Desktop/MAI_OS/lab3$
```

Рис. 2 – Зависимость времени от числа потоков

По полученным в ходе эксперимента данным были рассчитаны ускорение и эффективность работы системы. Результаты приведены в следующих таблицах и графиках.

Потоки	Время	Ускорение	Эффективность
1	2,25	1,00	100
2	1,08	2,07	103
3	0,73	3,06	76,5
4	0,61	3,66	45,7

Табл.1 – Сравнительная таблица

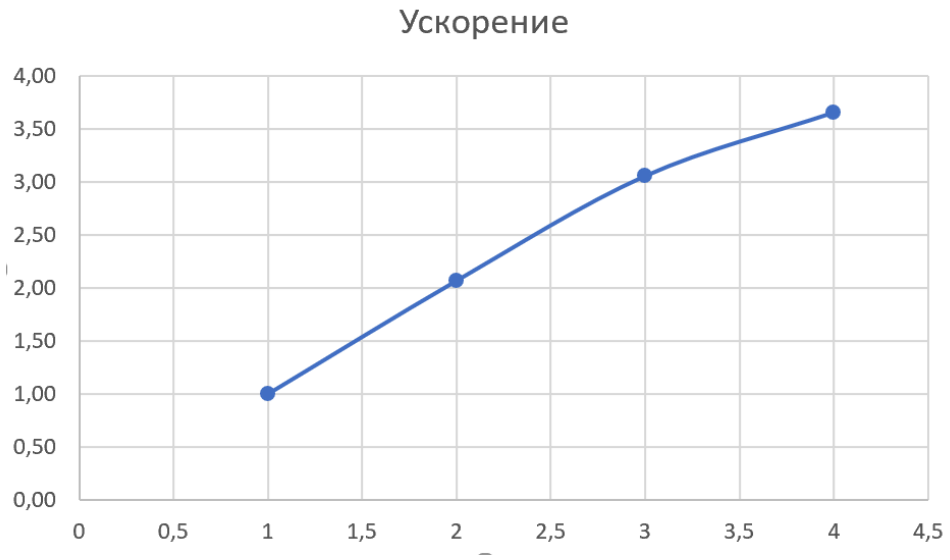


Рис. 3 – График зависимости ускорения от кол-ва потоков

3.4 Исследование зависимости времени от размера входных данных

При увеличении количества точек с 200 до 4000 время выполнения выросло с 0.001695 с до 5.412425с. Рост времени является нелинейным, что подтверждает теоретическую сложность алгоритма перебора всех троек точек. На малых данных время настолько мало, что накладные расходы на создание потоков и их синхронизацию могут быть сопоставимы с самим временем вычислений. На больших данных использование многопоточности дает существенный выигрыш по сравнению с последовательным выполнением. Результат приведён на рис.4.

```
student@DESKTOP-23PR51U:/mnt/c/Users/Admin/Desktop/MAI_OS/lab3$ ./triangle -t 4 -n 200
Thr: 4, N: 200
Max area: 7204.538247
Points: 103 148 181
Time: 0.001695 sec
student@DESKTOP-23PR51U:/mnt/c/Users/Admin/Desktop/MAI_OS/lab3$ ./triangle -t 4 -n 1000
Thr: 4, N: 1000
Max area: 7328.352692
Points: 204 218 400
Time: 0.137399 sec
student@DESKTOP-23PR51U:/mnt/c/Users/Admin/Desktop/MAI_OS/lab3$ ./triangle -t 4 -n 2000
Thr: 4, N: 2000
Max area: 7449.632038
Points: 103 388 1520
Time: 0.815138 sec
student@DESKTOP-23PR51U:/mnt/c/Users/Admin/Desktop/MAI_OS/lab3$ ./triangle -t 4 -n 4000
Thr: 4, N: 4000
Max area: 7623.768919
Points: 118 2466 3939
Time: 5.412425 sec
student@DESKTOP-23PR51U:/mnt/c/Users/Admin/Desktop/MAI_OS/lab3$
```

Рис. 4 – Зависимость времени работы от объёма данных

ВЫВОД

В ходе выполнения лабораторной работы были приобретены практические навыки управления потоками в операционной системе Linux с использованием библиотеки pthread. Была составлена и отлажена программа на языке Си, находящая треугольник максимальной площади в трехмерном пространстве.

В результате экспериментов было выявлено:

1. На малых входных данных (до 500 точек) использование 4 и более потоков не дает ускорения, а иногда даже замедляет программу. Это связано с накладными расходами на создание потоков, распределение ресурсов планировщиком ОС и их завершение (pthread_join).
2. На больших данных (2000+ точек) многопоточность дает существенный прирост. Однако ускорение не является линейным. При 4 потоках оно близко к 4-кратному, но эффективность падает из-за ограничений закона Амдала (последовательное сравнение локальных максимумов в конце).
3. При увеличении числа потоков до 8 эффективность падает, так как количество потоков превышает количество физических ядер процессора, из-за чего тратится время на переключение контекста.
4. Использование ключа запуска для ограничения потоков позволяет гибко управлять нагрузкой на систему и адаптировать программу под конкретное железо.

Все поставленные задачи выполнены в полном объеме.

ПРИЛОЖЕНИЕ

Triangle.c

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <math.h>
#include <time.h>
#include <getopt.h>

typedef struct {
    double x, y, z;
} Pt;

typedef struct {
    Pt *arr;
    int n;
    int id;
    int thr;
    double res;
    int i1, i2, i3;
} Task;

double area(Pt a, Pt b, Pt c) {
    double abx = b.x - a.x, aby = b.y - a.y, abz = b.z - a.z;
    double acx = c.x - a.x, acy = c.y - a.y, acz = c.z - a.z;
    double cx = aby * acz - abz * acy;
    double cy = abz * acx - abx * acz;
    double cz = abx * acy - aby * acx;
    return 0.5 * sqrt(cx * cx + cy * cy + cz * cz);
}

void* work(void *arg) {
    Task *t = (Task *)arg;
    t->res = -1.0;

    for (int i = t->id; i < t->n - 2; i += t->thr) {
        for (int j = i + 1; j < t->n - 1; j++) {
            for (int k = j + 1; k < t->n; k++) {
                double s = area(t->arr[i], t->arr[j], t->arr[k]);
                if (s > t->res) {
                    t->res = s;
                    t->i1 = i;
                    t->i2 = j;
```

```
                    t->i3 = k;
                }
            }
        }
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    int thr = 1;
    int n = 0;
    int key;

    while ((key = getopt(argc, argv, "t:n:")) != -1) {
        if (key == 't') thr = atoi(optarg);
        if (key == 'n') n = atoi(optarg);
    }

    if (thr < 1 || n < 3) {
        printf("Error: need thr >= 1 and n >= 3\n");
        return 1;
    }
    if (thr > n - 2) thr = n - 2;

    srand(42);
    Pt *pts = malloc(n * sizeof(Pt));
    for (int i = 0; i < n; i++) {
        pts[i].x = (double)rand() / RAND_MAX * 100.0;
        pts[i].y = (double)rand() / RAND_MAX * 100.0;
        pts[i].z = (double)rand() / RAND_MAX * 100.0;
    }

    pthread_t *tids = malloc(thr * sizeof(pthread_t));
    Task *tasks = malloc(thr * sizeof(Task));
    struct timespec st, en;
    clock_gettime(CLOCK_MONOTONIC, &st);
    for (int i = 0; i < thr; i++) {
        tasks[i].arr = pts;
        tasks[i].n = n;
        tasks[i].id = i;
        tasks[i].thr = thr;
        pthread_create(&tids[i], NULL, work, &tasks[i]);
    }
    for (int i = 0; i < thr; i++) {
        pthread_join(tids[i], NULL);
```

```
        }
        clock_gettime(CLOCK_MONOTONIC, &en);
        double dt = (en.tv_sec - st.tv_sec) + 1e-9 * (en.tv_nsec -
st.tv_nsec);
        double best = -1.0;
        int a = -1, b = -1, idx3 = -1;
        for (int i = 0; i < thr; i++) {
            if (tasks[i].res > best) {
                best = tasks[i].res;
                a = tasks[i].i1;
                b = tasks[i].i2;
                idx3 = tasks[i].i3;
            }
        }
        printf("Thr: %d, N: %d\n", thr, n);
        printf("Max area: %.6f\n", best);
        printf("Points: %d %d %d\n", a, b, idx3);
        printf("Time: %.6f sec\n", dt);
        free(pts);
        free(tids);
        free(tasks);
        return 0;
    }
```